

人工智能实践-数字经济应用实践 机器学习篇

主讲：高刚毅

邮箱：ggy0222@sina.com

部门：信智学院 人工智能系 9-411

Content

01 | 机器学习概述

02 | Sklearn 简介

03 | 模型调参

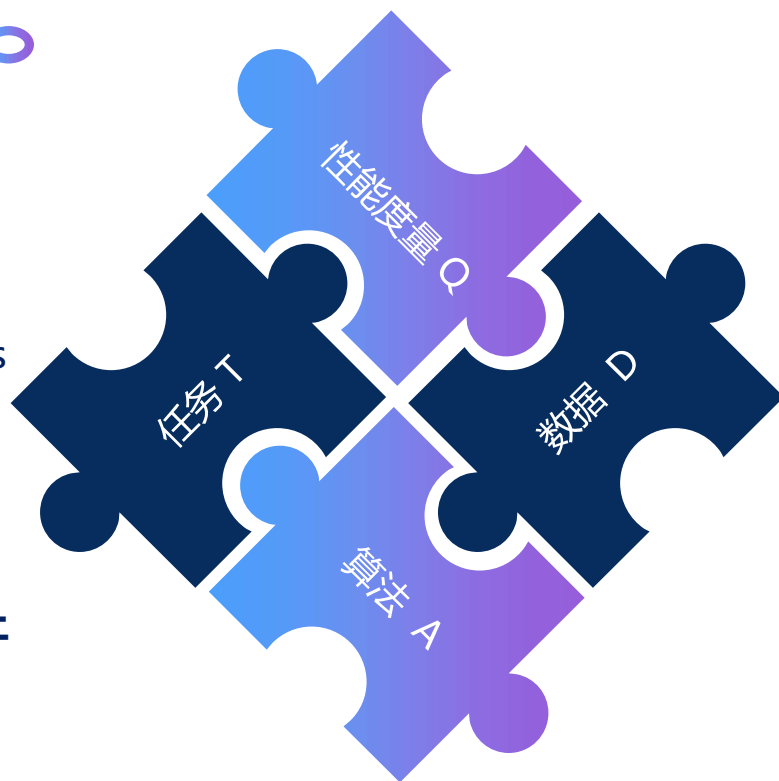
04 | 阶段考核要求

1、机器学习概述

机器学习的定义

A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P if its performance at tasks in T , as measured by P , improves with experience E .

定义：给定任务 T 和性能度量 Q ，建立算法 A 从数据 D 中学习，如果在任务 T 上的性能度量 Q 随着数据 D 增多而改善，那么可称机器在学习。



机器学习的任务

- **有监督学习：**从给定的已标注的训练数据集中学习一个模型，用新数据对这个模型预测结果。
- **无监督学习：**从无标注的训练数据集中学习并推断出数据的内在结构。
- **半监督学习：**介于监督学习与无监督学习之间的一种学习方式，主要考虑用少量的标注样本和大量的未标注样本进行训练和分类的问题。
- **强化学习：**通过观察来学习动作的完成，以一种试错的方式进行学习，找到最优策略。

机器学习应用

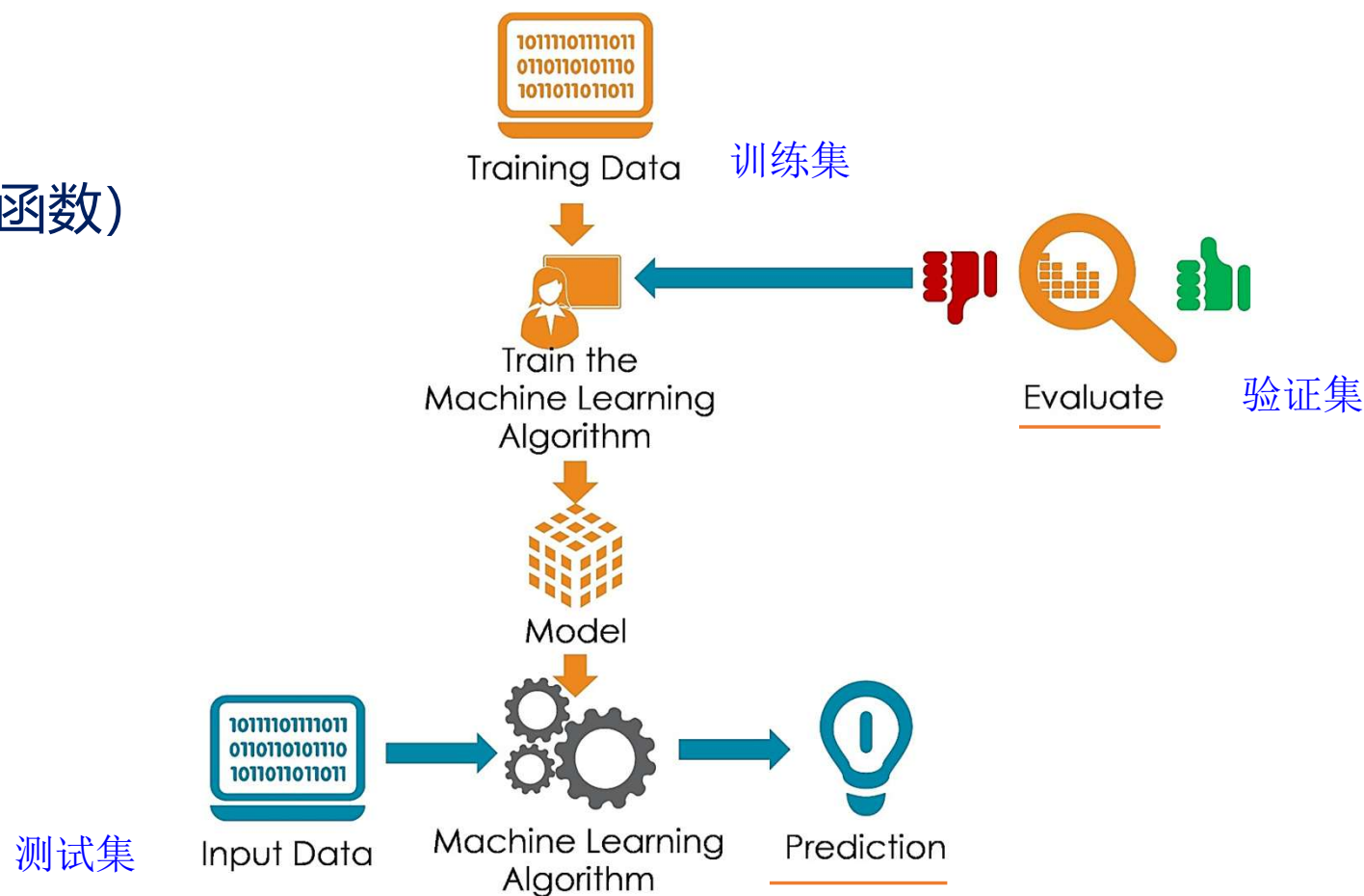
- 机器学习已经广泛应用于数据挖掘、**计算机视觉**、**自然语言处理**、统计学习、生物特征识别、搜索引擎、医学诊断、**欺诈检测（风控）**、证券市场分析、DNA序列测序、语音识别、模式识别、战略游戏和**机器人**领域。



- 监督学习**适用于**有明确标签**数据的预测任务，如垃圾邮件过滤、图像识别和语音识别；
- 无监督学习**用于发现**无标签**数据中的隐藏模式，如客户细分、聚类分析和异常检测；
- 半监督学习**结合少量标签数据与大量无标签数据，常用于医学图像分析等**标注成本高的**场景；
- 强化学习**则适用于智能体通过与环境**交互学习**最优策略的任务，如AlphaGo下棋、机器人控制等。

机器学习的一般流程

1. 模型选择
2. 确定目标函数（损失函数）
3. 模型训练和评估
 - 参数调优
4. 模型预测



模型选择的关键步骤

1. 明确任务类型、分析数据特性

- 是回归、分类、还是聚类问题？

2. 构建候选模型池

- 不同算法或同一算法的不同配置

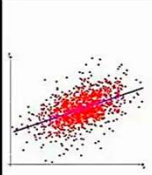
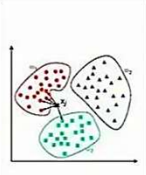
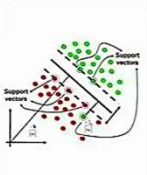
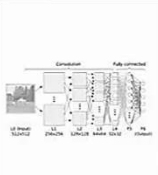
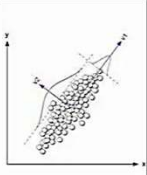
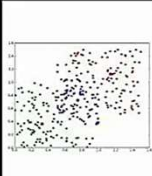
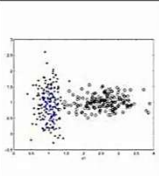
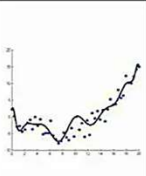
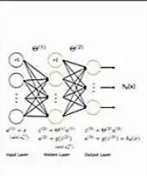
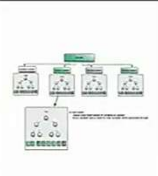
3. 设定评估指标

- 回归任务常用：均方误差（MSE）、 R^2 分数
- 分类任务常用：准确率、F1 分数、AUC-ROC

4. 训练、评估验证策略

- 训练集、验证集的划分，交叉验证策略等

5. 调优超参数（网格搜索等）

回归算法	决策树算法	基于实例算法	基于核的算法	深度学习	降低维度算法
					
聚类算法	贝叶斯算法	正则化算法	人工神经算法	集成算法	关联规则算法
					

常见的机器学习库



ONNX

theano

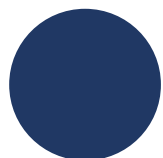


PYTORCH



易心Microbi编程

开发环境常用库



环境配置与安装:

- **Scikit-learn** 0.24.2 or 0.23.1
- **Python** 3.12.3 (Anaconda for python)
- **Numpy** 1.19.5, **Pandas** 1.1.5 , **SciPy** 1.6.0
- **Graphviz** 0.16
- **Matplotlib** 3.3.4, **seaborn** 0.9.0
- **Xgboost** 1.4.2
- **Pytorch**、**TensorFlow**、**Paddle**



2、Sklearn 简介

- **Scikit-learn** 涵盖了几乎所有主流机器学习算法，并且提供了一致的调用接口，基于 Numpy 和 Scipy 等 Python 数值计算库，提供了高效的算法实现。
- Python 标准库默认不包含 scikit-learn，需要自己下载安装。
- 安装命令：
 - `pip install scikit-learn`



Advanced Scikit-Learn

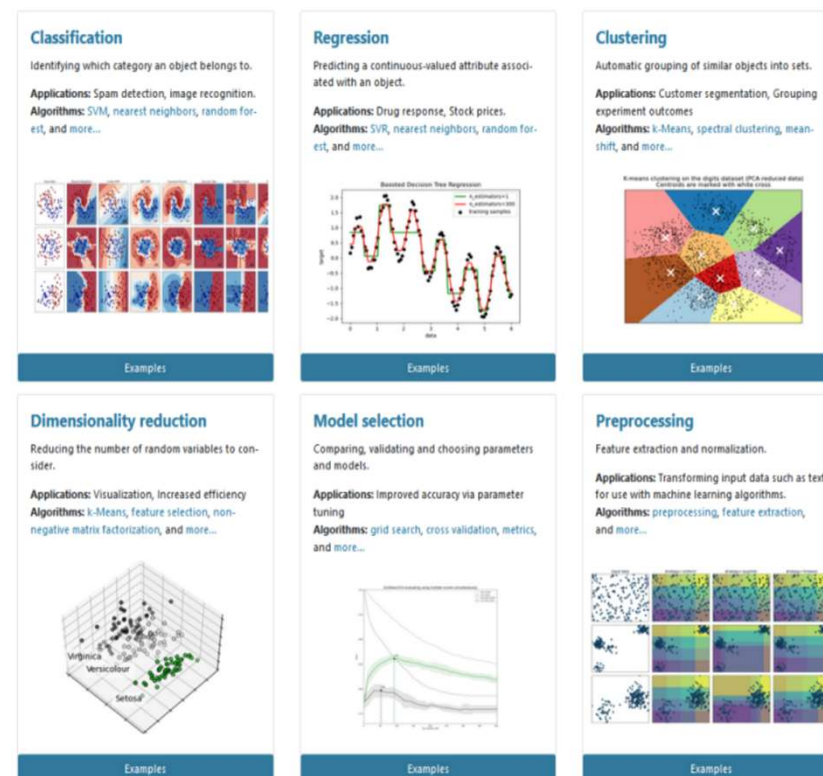
Andreas Mueller (NYU Center for Data Science, scikit-learn)

Scikit-learn 的主要功能模块

在 Sklearn 里面有六大任务模块：分别是分类、回归、聚类、降维、模型选择和预处理。

Scikit-learn 中的主要子库:

- ❑ datasets: 内置数据集
- ❑ preprocessing: 数据预处理,
- ❑ feature_selection: 过滤法、嵌入法、包装法
- ❑ model_selection: 数据集拆分、交叉验证、网格搜索等
- ❑ metrics: 分类和回归指标、混淆矩阵等
- ❑ linear_model: 线性回归、岭回归、逻辑回归、感知机等
- ❑ cluster: Kmeans、DBSCAN、GaussianMixture等
- ❑ tree: DecisionTreeClassifier、DecisionTreeRegressor
- ❑ svm: SVC、SVR、LinearSVC等
- ❑ ensemble: Bagging、RF、GBDT、AdaBoost等
- ❑ neural_network: MLPClassifier、MLPRegressor
- ❑ ...



Scikit-learn官网: <https://scikit-learn.org/stable/index.html>

Sklearn 分类模型

➤ Sklearn 库常用分类算法函数

- ❑ Sklearn 中提供的分类算法非常多，分别存在于不同的模块中。常用的分类算法如下表所示
- ❑ 分类模型的 Sklearn API 文档链接：https://scikit-learn.org/stable/supervised_learning.html#supervised-learning

模块名称	函数名称	算法名称
linear_model	LogisticRegression	逻辑回归
svm	SVC	支持向量机分类
neighbors	KNeighborsClassifier	k最邻近分类
tree	DecisionTreeClassifier	分类决策树
neural_network	MLPClassifier	多层感知机分类器
ensemble	RandomForestClassifier	随机森林分类器，并行
ensemble	GradientBoostingClassifier	梯度提升分类器
ensemble	AdaBoostClassifier	自适应提升分类器，串行

Sklearn 回归模型

➤ Sklearn 库常用回归算法函数

- ❑ 可以利用预测结果和真实结果画出折线图作对比，以便更直观看出线性回归模型效果。
- ❑ sklearn内部提供了不少回归算法，常用的函数如下表所示；
- ❑ 回归模型的 Sklearn API 文档链接：https://scikit-learn.org/stable/supervised_learning.html#supervised-learning

模块名称	函数名称	算法名称
linear_model	LinearRegression	线性回归
svm	SVR	支持向量机回归
neighbors	KNeighborsRegressor	最近邻回归
tree	DecisionTreeRegressor	回归决策树
ensemble	RandomForestRegressor	随机森林回归
ensemble	GradientBoostingRegressor	梯度提升回归
ensemble	AdaBoostRegressor	自适应提升回归

Sklearn 聚类模型

➤ Sklearn 库常用聚类算法函数

- ❑ Sklearn 常用的聚类算法模块 **cluster** 提供的聚类算法及其适用范围如下所示
- ❑ 聚类模型的 Sklearn API 文档链接: <https://scikit-learn.org/stable/modules/classes.html#module-sklearn.cluster>

函数名称	算法名称	参数	适用范围	距离度量
cluster.KMeans()	K均值聚类	簇数	可用于样本数目很大, 聚类数目中等的场景	点之间的距离
cluster.SpectralClustering()	将聚类应用于归一化拉普拉斯算子的投影	簇数	可用于样本数目中等, 聚类数目较小的场景	图距离
cluster.AgglomerativeClustering()	凝聚聚类	簇数, 链接类型, 距离	可用于样本数目很大, 聚类数目较大的场景	任意成对点线图之间的距离
cluster.DBSCAN()	从向量数组或距离矩阵进行DBSCAN聚类。	半径大小, 最低成员数目	可用于样本数目很大, 聚类数目中等的场景	最近的点之间的距离
cluster.Birch()	Birch层次聚类	分支因子, 阈值, 可选全局, 集群	可用于样本数目很大, 聚类数目中等的场景	点之间的欧式距离

分类模型参数介绍: LogisticRegression

LogesticRegression

```
class sklearn.linear_model.LogisticRegression (penalty=' l2' , dual=False, tol=0.0001, C=1.0,  
fifit_intercept=True, intercept_scaling=1, class_weight=None, random_state=None, solver=' warn' ,  
max_iter=100,  
multi_class=' warn' , verbose=0, warm_start=False, n_jobs=None)
```

重要参数:

- 1.**penalty**: 正则化方式, 可以输入" l1" 和 " l2" , 默认是" l2" 。如果用l1正则化, 参数solver采用" liblinear" 和" saga" , 如果用" l2" 正则化, 参数solver可用所有的求解方式。
 - 2.sovler: 用于求解模型最优化参数的算法, 可以输入{ "newton-cg" , " lbfgs" , "liblinear" , " sag" }, 默认为" liblinear" 。
 - 3.max_iter: 整数, 默认100。求解器收敛的最大迭代次数, 仅适用于newton-cg, sag和lbfgs求解器。
 - 4.C: C正则化强度的倒数, 必须是一个大于0的浮点数, 不填写默认为1.0。默认正则项与损失函数的比值是1:1。C越小, 损失函数就会越小。C越小, 损失函数就会越小, 模型对损失函数的惩罚就越重。正则化效力就越强。
- 接口: fit, predict, predict_proba, score

分类模型参数介绍: Decision Tree

Decision Tree

```
class sklearn.tree.DecisionTreeClassifier (criterion=' gini' , splitter=' best' , max_depth=None,
min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features=None,
random_state=None, max_leaf_nodes=None, min_impurity_decrease=0.0, min_impurity_split=None,
class_weight=None, presort=False)
```

重要参数:

- 1.**criterion**: 用来决定不纯度的计算方法, 提供2种选择: **信息熵 “entropy”** 和**基尼系数 “gini”** , 默认为 “gini” 。信息熵很容易过拟合, 基尼系数在这种情况下效果往往比较好。
- 2.**max_depth**: 剪枝参数, 限制**树的最大深度**, 超过设定深度的树枝全部剪掉。可有效防止过拟合, 建议设置从3开始。
- 3.**min_samples_leaf**: 设置**叶子结点上的最小样本数**。设置太小易过拟合, 太大会阻止模型学习。建议从5开始。
- 4.**min_samples_split**: 当对一个内部结点划分时, 要求该**结点上的最小样本数**, 否则不再划分。
- 5.**splitter**: 控制决策树的随机选项的, 有两个取值 “ best ” 和 “ random ” , “ best ” 优先选择更重要的特征来进行分枝, “ random ” 决策树在分枝更加随机。
- 6.**max_features**: 限制分枝时考虑的特征个数, 超过限制个数的特征都会被舍弃。在不知道各个特征重要性的情况下, 强行设置这个参数可能会导致模型学习不足。
- 7.**min_impurity_decrease**: 限制信息增益的大小, 信息增益小于设定数值的分枝不会发生。

分类模型参数介绍：SVM

SVM

```
class sklearn.svm.SVC (C=1.0, kernel=' rbf' , degree=3, gamma=' auto_deprecated' ,  
coef0=0.0, shrinking=True,  
probability=False, tol=0.001, cache_size=200, class_weight=None, verbose=False,  
max_iter=-1,  
decision_function_shape=' ovr' , random_state=None)
```

重要参数：

- 1.C：错误项的**惩罚系数**。C越大，即对分错样本的惩罚程度越大，因此在训练样本中准确率越高，但是泛化能力降低。相反，减小C的话，容许训练样本中有一些误分类错误样本，泛化能力强。
- 2.Kernel：算法中采用的**核函数类型**，默认为" rbf" 。可选的参数：linear, poly, rbf, sigmoid, precomputed（核矩阵）。
- 3.Degree：int型参数，默认为3，只对多项式核函数有用。
- 4.Gamma：核函数系数，默认为auto，只对" rbf" ," poly" ," sigmoid" 有效。
- 5.max_iter：int型参数，默认为-1.最大迭代数，如果为-1，表示不限制。

分类模型参数介绍: KNN

KNN

```
sklearn.neighbors.KNeighborsClassifier(n_neighbors=5, weights=' uniform' ,  
algorithm=' auto' , leaf_size=30, p=2, metric=' minkowski' , metric_params=None,  
n_jobs=None, **kwargs)
```

重要参数:

1.**n_neighbors**: KNN中的 **k 值**, 默认为5。

2.**Weight**: 用于每个标识样本的近邻样本的权重, 可以选择 uniform 和 distance 或者自定义权重。默认为" uniform" ,所有最近邻样本权重都是一样。如果是distance, 则权重和距离成反比。

3.**algorithm**: 限定半径最近邻法使用的算法, 可选 'auto' , 'ball_tree' , 'kd_tree' , 'brute' 。

'brute' 对应第一种线性扫描;

'kd_tree' 对应第二种kd树实现;

'ball_tree' 对应第三种球树实现;

'auto' 则会在上面三种算法中做权衡, 选择一个拟合最好的最优算法。

4.**leaf_size**: 这个值控制了使用kd树或者球树时, 停止建子树的叶子节点数量的阈值。这个值越小, 则生成的kd树或者球树就越大, 层数越深, 建树时间越长, 反之, 则生成的kd树或者球树会小, 层数较浅, 建树时间较短。默认是30

分类模型参数介绍: Random Forest

Random Forest

class sklearn.ensemble.**RandomForestClassifier**

```
(n_estimators=' 10' , criterion=' gini' , max_depth=None,  
min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features=' auto' ,  
max_leaf_nodes=None, min_impurity_decrease=0.0, min_impurity_split=None, bootstrap=True,  
oob_score=False,  
n_jobs=None, random_state=None, verbose=0, warm_start=False, class_weight=None)
```

重要参数:

- 1.**n_estimators**: 随机森林中**树的个数**, 即要生成多少个基学习器 (决策树)
- 2.**criterion**: 选择**最优划分属性的准则**, 默认 "gini" 基尼系数可选信息熵 "entropy"
- 3.**max_depth**: **树的最大深度**, 超过最大深度的树枝都会被剪掉
- 4.**min_samples_leaf**: 设置**叶子结点上的最小样本数**, 否则分枝就不会发生;
- 5.**min_samples_split**: 当对一个内部结点划分时, 要求该**结点上的最小样本数**, 否则不再划分;
- 6.**max_features**: 限制分枝时考虑的特征个数, 超过限制个数的特征都会被舍弃, 默认值为总特征个数开平方取整, 如0.2允许利用特征数为20%;
- 7.**bootstrap**: 是否采用**自助式采样**生成采样集。

分类模型参数介绍: XGBOOST

XGBOOST (eXtreme Gradient Boosting)

```
class xgboost.XGBRegressor (max_depth=3, learning_rate=0.1, n_estimators=100, silent=True,  
objective='reg:linear', booster='gbtree', n_jobs=1, nthread=None, gamma=0, min_child_weight=1,  
max_delta_step=0,  
subsample=1, colsample_bytree=1, colsample_bylevel=1, reg_alpha=0, reg_lambda=1,  
scale_pos_weight=1,  
base_score=0.5, random_state=0, seed=None, missing=None, importance_type='gain', **kwargs)
```

重要参数:

- 1.**n_estimators**: 森林中**树的数量**, 初始越多越好, 但是会增加训练时间, 到达一定数量后模型的表现不会再有显著的提升。
- 2.**max_depth**: **树的最大深度**。
- 3.**learning_rate**: **学习率**。
- 4.**max_features**: 各个基学习器进行切分时随机挑选的特征子集中的特征数目, 数目越小模型整体的方差会越小, 但是单模型的偏差也会上升, 经验性的设置回归问题的max_features为整体特征数目, 而分类问题则设为整体特征数目开方的结果。

分类模型参数介绍：XGBOOST

XGBOOST

其他参数：

`min_samples_split`: 节点最小分割的样本数，表示当前树节点还可以被进一步切割的含有的最少样本数；经验性的设置为1，原因同上

`bootstrap`, `rf`里默认是`True`也就是采取自助采样，而`Extra-Trees`则是默认关闭的，是用整个数据集的样本，当`bootstrap`开启时，同样可以设置`oob_score`为`True`进行包外估计测试模型的泛化能力

`n_jobs`: 并行化，可以在机器的多个核上并行的构造树以及计算预测值，不过受限于通信成本，可能效率并不会说分为`k`个线程就得到`k`倍的提升，不过整体而言相对需要构造大量的树或者构建一棵复杂的树而言还是高效的

`criterion`: 切分策略，`gini` 或者 `entropy`，默认是`gini`，与树相关。

`min_impurity_split`→`min_impurity_decrease`:用来进行早停止的参数，判断树是否进一步分支，原先是比较不纯度是否仍高于某一阈值，0.19后是判断不纯度的降低是否超过某一阈值

`warm_start`: 若设为`True`则可以再次使用训练好的模型并向其中添加更多的基学习器

`class_weight`: 设置数据集中不同类别样本的权重，默认为`None`,也就是所有类别的样本权重均为1，数据类型为字典或者字典列表(多类别)

`balanced`: 根据数据集中的类别的占比来按照比例进行权重设置

Sklearn 的神经网络模型

- 在 `scikit-learn` (sklearn) 中，深度学习模型的实现相对有限，主要聚焦于基础的神经网络结构。主要模型有：
 - `MLPClassifier`: 用于分类任务的多层感知机，支持自定义隐藏层结构、激活函数（如 ReLU、Sigmoid）和优化器（如 Adam、SGD）。
 - `MLPRegressor`: 用于回归任务的多层感知机，输出为连续值。
- 示例：
 - `from sklearn.neural_network import MLPClassifier`
 - `clf = MLPClassifier(hidden_layer_sizes=(100,), activation= 'relu' ,
solver= 'adam' , alpha=1e-4)` #创建网络模型
 - `clf.fit(X_train, y_train)` #训练模型

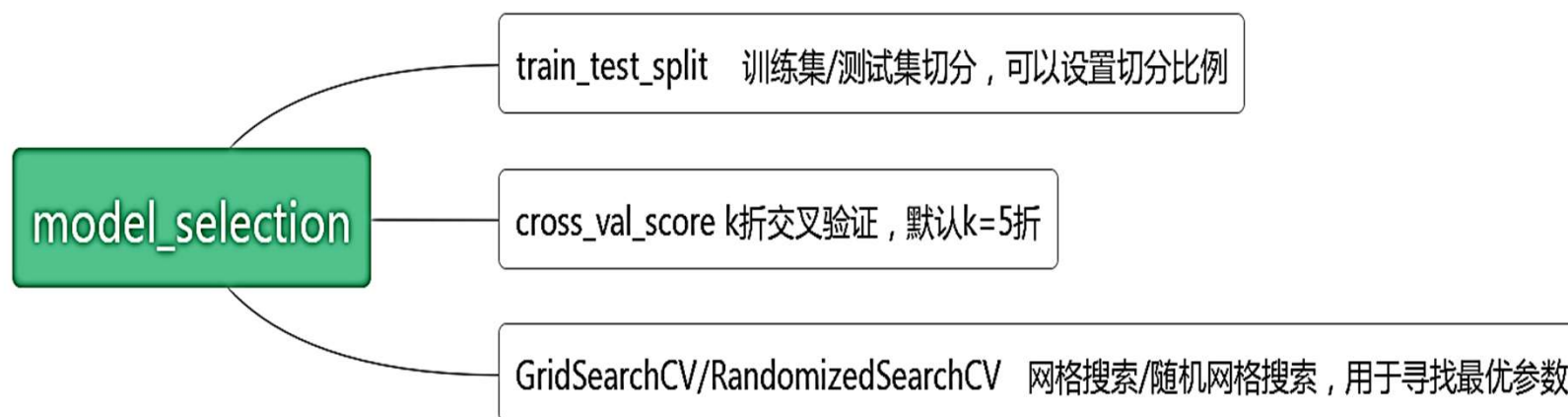
模型度量指标

- 评价指标针对不同的机器学习任务有不同的指标，同一任务也有不同侧重点的评价指标。主要有分类（classification）、回归（regression）、聚类（clustering）等。
- 模块导入方式：**sklearn.metrics**



模型训练与评估

- 模型训练与评估是机器学习中的重要环节，涉及到的操作包括数据集切分、模型创建、训练、评估（调参）等。对应常用函数包括：



样本类别不平衡

- **样本类别不平衡**：是指在分类任务中，不同类别的训练样本数量存在显著差异的现象。通常负样本很多，负样本在总的损失函数中的占比很大，导致最终模型对正样本的分类效果较差。
- 解决方法：
 - 1、**上采样/过采样** (Under-sampling)
 - 通过采样对样本进行扩充，简单的重复样本不是好方法，可以采用**数据增强**的方式。
 - **安装命令**：pip install imblearn
 - from imblearn.over_sampling import **SMOTE**
 - 2、**下采样/欠采样** (Over-sampling)
 - 通过**随机采样**、**分层采样**、**加权采样**等方式对样本进行筛选，保留比较有代表性的样本，去掉大量重复的相似的样本。
 - from imblearn.under_sampling import **RandomUnderSampler**

模型训练与评估：k折交叉验证

- **K-Fold cross-validation**: (**k折交叉验证**)，通常K取10折，相当于把数据集平均切分为10份，并逐一选择其中一份作为测试集、其余作为训练集进行训练及评分，最后返回K个评分的均值。



sklearn 相关函数

- `sklearn.model_selection.train_test_split(*arrays, test_size=None, train_size=None, random_state=None, shuffle=True, stratify=None)`
 - 数据集拆分：将所有数据分成**训练集**和**测试集**两部分。
 - 缺点：数据只划分了一次，且划分具有**偶然性**。
- `sklearn.model_selection.cross_val_score(estimator, X, y=None, *, groups=None, scoring=None, cv=None, n_jobs=None, verbose=0, fit_params=None, pre_dispatch='2*n_jobs', error_score=nan)`
 - k折交叉验证：进行k次的训练集和测试集的划分，评估结果一般取平均值。从而提高其泛化能力。

3、模型调参



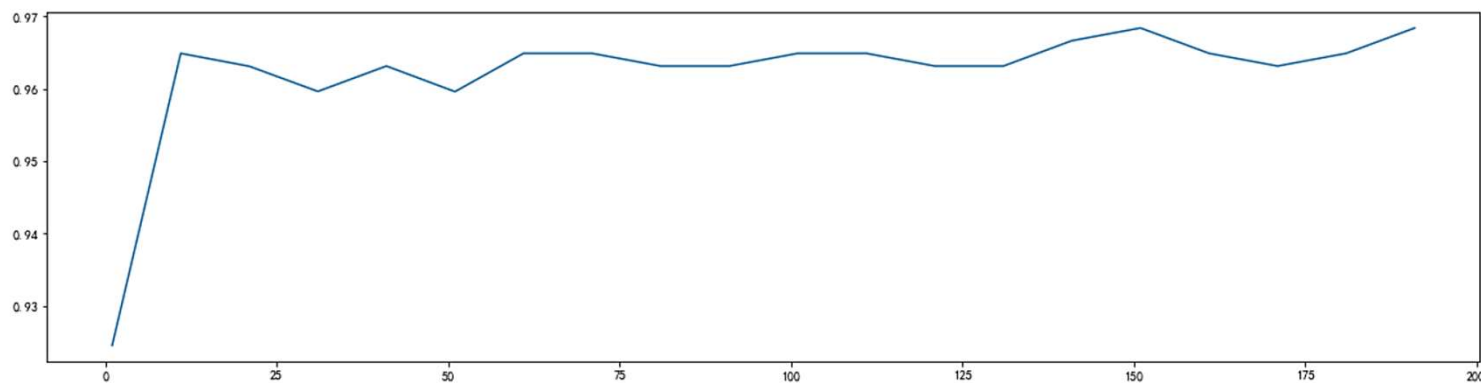
- **什么是模型调参?** 是为了找到最优的超参数组合。
 - 超参数 (如学习率、树的深度、正则化系数) 是在训练前设定的, 它们不通过数据学习, 但决定了模型如何学习。
 - 调参的本质是在模型复杂度与泛化能力之间寻找平衡点。模型太复杂会过拟合, 模型太简单会欠拟合。

常用方法:

1. 学习曲线 (Learning Curve)
2. 网格搜索 (GridSearchCV)
3. 随机搜索 (RandomizedSearchCV)

3、模型调参

- 1、学习曲线调参 (Learning Curve)
 - 参数学习曲线是一条以不同的参数取值为横坐标，不同参数取值下的模型评估结果为纵坐标的曲线。
 - 我们可以选择模型表现最佳点的参数取值为参数赋值。



3、模型调参

• 2、网格搜索调参

- GridSearchCV (网络搜索交叉验证) 用于系统地遍历模型的多种参数组合, 通过交叉验证从而确定最佳参数, 适用于小数据集。
- `class sklearn.model_selection.GridSearchCV(estimator, param_grid, *, scoring=None, n_jobs=None, refit=True, cv=None, verbose=0, pre_dispatch='2*n_jobs', error_score=nan, return_train_score=False)`
 - 参数: `estimator`: 选择的分类器; `param_grid`: 需要优化参数的取值, 类型为字典;
 - `cv`: 交叉验证参数, 默认为 None; `scoring`: 模型准确率评价标准, 默认是 None;
 - 属性: `best_estimator_`: 最佳模型; `best_score_`: 最佳模型下的分数;
`best_params_`: 最佳模型参数; `cv_results_`: 具体用法模型不同参数下交叉验证的结果。

3、模型调参

- 3、随机搜索 RandomizedSearchCV
- 采用网格搜索当超参数个数比较多的时候，搜索时间将会指数级上升。因此有人就提出了随机搜索的方法，随机在超参数空间中搜索几十几百个点，提高了搜索速度。
- `class sklearn.model_selection.RandomizedSearchCV(estimator, param_distributions, *, n_iter=10, scoring=None, n_jobs=None, refit=True, cv=None, verbose=0, pre_dispatch='2*n_jobs', random_state=None, error_score=nan, return_train_score=False)`
 - 参数：`estimator`：选择的分类器；`param_distributions`：定义超参数的搜索空间；
 - `n_iter`：随机采样的参数组合数量；`cv`：交叉验证策略。默认为 5；
- 注意：在实践中，人们不会使用网格搜索同时搜索这么多不同的参数，而是只选择那些被认为最重要的参数。

模型的保存与调用

□ Pickle保存:

Pickle是python编程中比较标准的一个保存和调用模型的库，我们可以使用pickle和open函数的连用，来将我们的模型保存到本地。



步骤:

1. 载入模块 `import pickle`
2. 保存模型
`with open('model.pickle','wb') as f:`
`pickle.dump(model,f)`
3. 加载模型
`with open('model.pickle','rb') as f:`
`model_get = pickle.load(f)`

□ Joblib保存:

Joblib是SciPy生态系统中的一部分，它为Python提供保存和调用管道和对象的功能，处理NumPy结构的数据尤其高效，对于很大的数据集和巨大的模型非常有用。



步骤:

1. 载入模块 `import joblib`
2. 保存模型
`joblib.dump(model,'filename.pkl')`
3. 加载模型
`model = joblib.load('filename.pkl')`

4、阶段考核要求

- **阶段目标**：综合对比分析不同算法模型，完成模型的选择、训练和评估。
- **主要任务可包括**：
 1. **数据集划分**：训练集、验证集。另外，有专门的测试集。
 2. **样本分类不平衡问题处理**：欠采样和过采样
 3. **模型选择、训练与评估**：
 - 综合运用传统机器学习和神经网络进行模型的选择、训练和评估，给出最终模型选择结果，并保存模型文件；
 4. **模型调参**：
 - 综合运用模型调参的基本方法：如学习曲线、网格搜索、随机搜索等；
 - 调参过程可适当进行可视化；欢迎使用其他深度学习框架进行训练和调参。
- **答辩展示**：建议采用 Jupyter Notebook 分步骤展示所做工作。

The image features a dark blue background with a complex, glowing circuit board pattern. In the center, there is a dark blue, rounded cloud-like shape. Inside this shape, the Chinese text "谢谢观看!" (Thank you for watching!) is written in a bold, white, sans-serif font. Several bright blue lines, resembling data streams or circuit traces, enter the cloud from the top and exit from the bottom, creating a sense of connectivity and flow.

谢谢观看!